End-to-End Deployment Cheatsheet

1. What is Deployment?

Putting your app/website on a machine (server) reachable over the internet — not just `localhost:3000`.

Two hosting options:

- **Self-hosted**: your own laptop/PC as server (needs 24/7 power + stable internet — impractical)
- **Cloud platforms**: DigitalOcean, Render, Vercel, Netlify, Railway, AWS, GCP, Azure

---

2. Scaling Levels (by user count)

Level	Users	RPS	What Changes	Tools
1	0 – 1K	1–20	Single server, minimum cost, basic deploy	Render, DigitalOcean (low-cost VM)
2	1K – 10K	20–400	Handle concurrency, optimize DB connections	DO VPS, Render
3	10K – 100K	100–1K	Load balancers, DB replication, caching, server replicas	AWS, GCP, Azure, Redis
4	100K – 1M	1K+	"Real engineering" — DB sharding, distributed monolith	AWS/GCP + Kubernetes, Kafka, distributed caching
5	1M+	Very high	Multi-region infra, bare metal, custom cloud deals	AWS/GCP + Kubernetes + advanced message brokers

Rule of thumb: Start simple (Level 1–2 tools), migrate up only when metrics force you to.

---

3. Concurrency vs RPS

- **Concurrency** = active users connected to server *right now*
- **RPS (Requests/sec)** = actual requests hitting server per second
- **RPS ≈ 2–5% of concurrency** (most connected users aren't actively firing requests every second)

---

4. Platform Comparison

Platform	Beginner-Friendly	Free Tier	Best For	Notes
Render	Yes	Yes (~1L free requests)	Beginners, small apps	Easy repo-based deploy
DigitalOcean	Moderate	No	VPS, small–medium workloads	Low-cost VMs, manual config
Vercel / Netlify	Yes	Yes	Static sites, frontend	Serverless backend support
AWS (EC2/Lambda)	No	Yes (limited)	Enterprise, large-scale	Rich ecosystem, complex
GCP	Moderate	Yes	AI/ML/data workloads	Gemini AI integration
Azure / Oracle	No	—	Enterprise clients	Enterprise-focused

5. Common Bottlenecks & Fixes

Problem	Cause	Fix
Single point of failure	One server, no redundancy	Load balancer + replicas
DB bottleneck	Too many connections	Caching (Redis), sharding, replication
High CPU/RAM usage	Poorly written code	Optimize code, microservices for heavy tasks
Network latency / outages	Data traveling across regions	Multi-region deployment
Traffic scaling issues	Sudden growth, no plan	Ship MVP fast, scale reactively not preemptively

6. Key Techniques

- **Load Balancing** — distribute traffic across multiple servers
- **Caching (Redis)** — reduce DB hits for repeated reads
- **Sharding** — split DB into partitions for speed + scale
- **Replication** — copies of DB for read scaling/failover
- **Microservices** — split app into independent, scalable services
- **Design for Failure** — system stays up even if one service fails
- **Message Brokers (Kafka)** — decouple services, handle async load at scale

---

7. Golden Rules

1. **Level 1–2:** Speed > perfection. Ship the MVP on Render/DigitalOcean.
2. **Level 3+:** Add load balancers, caching, replication before you need them — not after outages.
3. **Level 4–5:** Think distributed systems — sharding, Kubernetes, Kafka become mandatory.
4. Database is *usually* the first real bottleneck — cache and index before scaling compute.
5. Don't over-engineer early; don't under-engineer once traffic is real.
6. Cloud (AWS/GCP/Azure) > raw VM once you need autoscaling, managed connection pooling, and multi-region.

---

8. Glossary (Quick Recall)

`Deployment` · `Concurrency` · `RPS` · `Load Balancer` · `Sharding` · `Caching` · `Microservices` · `Bottleneck` · `Design for Failure` · `MVP` · `Message Broker`